

FORMATO DE ENTREGA DE EVIDENCIAS

Serie de guías · Desarrollo de APIs con FastAPI

1. Datos del aprendiz

Nombre completo	Joel Andrés Mendoza Buriticá
Número de documento	1042853355
Programa	Análisis y Desarrollo de Software (ADSO)
Ficha	3169892
Número de la guía	Guía 02
Nombre de la guía	APIS FastApi
Instructor	Mateo Arroyave
Fecha de entrega	28 / 07 / 2026

Cómo diligenciar este formato

Complete cada evidencia reemplazando el texto en gris. Escriba dentro de los recuadros y pegue las capturas donde se indica. Al terminar, exporte a PDF (Archivo ▶ Guardar como ▶ PDF) y suba el archivo a Google Classroom en la actividad correspondiente.

2. Enlace al repositorio de GitHub

Muchas evidencias piden el código en GitHub. Pegue aquí el enlace de su repositorio (debe ser público):

<https://github.com/JBUR10/APIS-FASTAPIS.git>

3. Evidencia 1 — Reflexión inicial

Código de la evidencia	GA2-AA1-EV01
Tipo	Conocimiento

Respuestas a las preguntas de reflexión:

1. Si un solo archivo llegara a tener 800 líneas, ¿qué problemas cree que aparecerían al buscar un error o al trabajar en equipo?

El problema inicial sería la dificultad de buscar exactamente lo que se necesita, también entender qué hace cada parte del archivo sería más complicado, cuando se agreguen cosas nuevas podría haber problemas en partes anteriores del código y tendría que volver allí (y encontrarlo) para resolverlo.

2. Cuando llena un formulario de registro y deja el correo vacío, la página se lo reclama. ¿Quién cree que debería validar que los datos lleguen completos y correctos: ¿el usuario, el front-end o el servidor?

El servidor es el responsable de validar esta información, ya que el usuario es el encargado de ingresar los datos, el frontend de recibirlos y evitar que se envíen datos vacíos.

3. En la Guía 01 solo podía consultar productos. ¿Qué operaciones le faltan a una tienda para ser realmente útil (piense en el día a día de un administrador)?

La capacidad de crear los productos, actualizarlos (esto incluye su stock, descripciones de producto, fotografías, etc.) eliminar un producto, administrar los usuarios, etc.

4. Las palabras Crear, Leer, Actualizar y Eliminar forman la sigla CRUD. ¿Con qué método HTTP asociaría cada una?

Crear es POST

Leer es GET

Actualizar es PUT

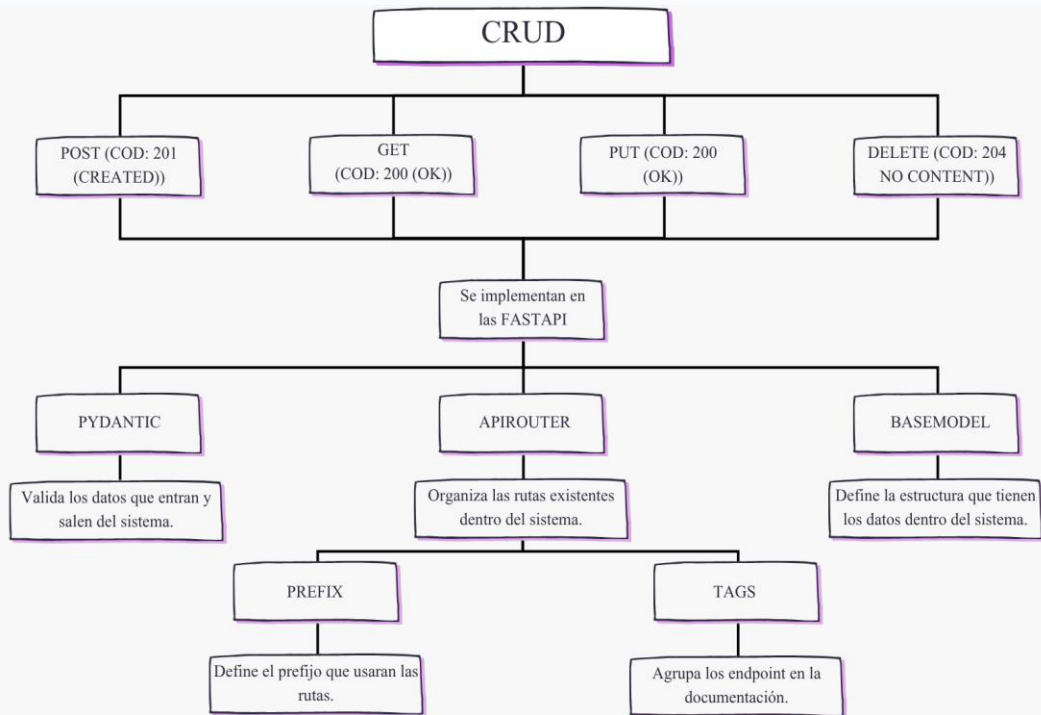
Eliminar es DELETE

4. Evidencia 2 — Contextualización

Código de la evidencia	GA2-AA1-EV02
Tipo	Conocimiento

Cuadro comparativo / mapa conceptual solicitado en la guía. Puede escribirlo en el recuadro o pegar una imagen de su cuadro/mapa:

Mapa conceptual sobre CRUD, pydantic, apirouter, basemodel, prefix y tags.



5. Evidencia 3 — Apropiación (laboratorio)

Código de la evidencia	GA2-AA2-EV03
Tipo	Desempeño

Descripción de lo que hizo: explique brevemente los pasos que siguió y los ejercicios que resolvió.

1. Desde el /docs cree dos productos usando categorías ya existentes, procedí luego a verificar su existencia con el GET que obtiene todos los productos.
2. Creo el producto y cambio el precio de este, luego, consulto el cambio con el id de ese producto.
3. Elimino un producto a través de su id luego verifico que se eliminó a través del GET.

Capturas de pantalla: pegue la evidencia de su API funcionando en /docs y de los endpoints probados.

1.

```
Code    Details
201
Response body
{
  "mensaje": "Producto creado",
  "producto": {
    "id": 4,
    "nombre": "TV 55\"",
    "precio": 800,
    "categoria": "Pantallas"
  }
}
Download

Code    Details
201
Response body
{
  "mensaje": "Producto creado",
  "producto": {
    "id": 5,
    "nombre": "Alcatel One Touch Pixi",
    "precio": 200,
    "categoria": "Pantallas"
  }
}

{
  "id": 4,
  "nombre": "TV 55\"",
  "precio": 800,
  "categoria": "Pantallas"
},
{
  "id": 5,
  "nombre": "Alcatel One Touch Pixi",
  "precio": 200,
  "categoria": "Pantallas"
}
}
```

2.

```
Code    Details
201
Response body
{
  "mensaje": "Producto creado",
  "producto": {
    "id": 5,
    "nombre": "TV 55\"",
    "precio": 800,
    "categoria": "Pantallas"
  }
}
```

3.

Code	Details
200	Response body <pre>{ "mensaje": "Producto actualizado", "producto": { "id": 5, "nombre": "TV 55'", "precio": 2, "categoria": "Pantallas" } }</pre>
200	Response body <pre>{ "id": 5, "nombre": "TV 55'", "precio": 2, "categoria": "Pantallas" }</pre>

Code	Details
200	Response body <pre>{ "mensaje": "Producto eliminado", "producto": { "id": 5, "nombre": "TV 55'", "precio": 2, "categoria": "Pantallas" } }</pre>

Request URL
<pre>http://127.0.0.1:8000/productos/5</pre>

Server response	
Code	Details
404 <i>Undocumented</i>	Error: Not Found Response body <pre>{ "detail": "Producto no encontrado" }</pre>

4.

Code	Details
404 <i>Undocumented</i>	Error: Not Found Response body <pre>{ "detail": "Producto no encontrado" }</pre>
Code	Details
422	Error: Unprocessable Content Response body <pre>{ "detail": [{ "type": "missing", "loc": ["body", "nombre"], "msg": "Field required", "input": {} }], {</pre>

6. Evidencia 4 — Transferencia (reto)

Código de la evidencia	GA2-AA3-EV04
Tipo	Producto

Enlace al repositorio con el reto resuelto:

<https://github.com/JBUR10/APIS-FASTAPIS.git>

Explique brevemente su solución:

1. Cree los 4 Endpoint basándome completamente en los endpoints de los productos, por lo que se me hizo mas sencillo, se me complica un poco las sintaxis diferentes de cada lenguaje, sin embargo, en base a lo anterior adapte estas a las categorías.
2. Probe los endpoints desde el docs confirmando que todos son funcionales.

Capturas del reto funcionando:

1.

```
@router.get("/{categoria_id}")
def obtener_categoria(categoria_id: int):
    for categoria in categorias:
        if categoria["id"] == categoria_id:
            return categoria
    raise HTTPException(status_code=404, detail="Categoria no encontrada")

@router.post("", status_code=201)
def crear_categoria(datos: CategoriaEntrada):
    nuevo_id = max([c["id"] for c in categorias], default=0) + 1
    nueva = {
        "id": nuevo_id,
        "nombre": datos.nombre
    }
    categorias.append(nueva)
    return {"mensaje": "Categoria creada", "categoria": nueva}

@router.put("/{categoria_id}")
def actualizar_categoria(categoria_id: int, datos: CategoriaEntrada):
    for categoria in categorias:
        if categoria["id"] == categoria_id:
            categoria["nombre"] = datos.nombre
            return {"mensaje": "Categoria actualizada", "categoria": categoria}
    raise HTTPException(status_code=404, detail="Categoria no encontrada")

@router.delete("/{categoria_id}")
def eliminar_categoria(categoria_id: int):
    for categoria in categorias:
        if categoria["id"] == categoria_id:
            categorias.remove(categoria)
            return {"mensaje": "Categoria eliminada", "categoria": categoria}
    raise HTTPException(status_code=404, detail="Categoria no encontrada")
```

Code	Details
200	Response body <pre>[{ "id": 1, "nombre": "Perifericos" }, { "id": 2, "nombre": "Pantallas" }, { "id": 3, "nombre": "Audio" }]</pre>

2.

Code	Details
201	Response body <pre>{ "mensaje": "Categoria creada", "categoria": { "id": 5, "nombre": "Juguetes" } }</pre>

Code	Details
200	Response body <pre>{ "id": 1, "nombre": "Perifericos" }</pre>

Code	Details
200	Response body <pre>{ "mensaje": "Categoria actualizada", "categoria": { "id": 5, "nombre": "Balones" } }</pre>

Code	Details
200	Response body <pre>{ "mensaje": "Categoria eliminada", "categoria": { "id": 5, "nombre": "Balones" } }</pre>

Code	Details
404 <i>Undocumented</i>	Error: Not Found Response body <pre>{ "detail": "Categoria no encontrada" }</pre>

Code	Details
422	Error: Unprocessable Content
	Response body
	<pre>{ "detail": [{ "type": "missing", "loc": ["body", "nombre"], "msg": "Field required", "input": {} }] }</pre>

7. Lista de chequeo (autoevaluación)

Antes de subir a Classroom, verifique que cumplió con todo. Marque cada casilla:

- ✓ ☐ Diligencíé mis datos (nombre, documento, ficha, número de guía).
- ✓ ☐ Respondí la Evidencia 1 — Reflexión inicial.
- ✓ ☐ Entregué la Evidencia 2 — Cuadro/mapa de contextualización.
- ✓ ☐ Entregué la Evidencia 3 — Laboratorio, con capturas de /docs.
- ✓ ☐ Entregué la Evidencia 4 — Reto, con enlace a GitHub y capturas.
- ✓ ☐ Mi repositorio de GitHub es público y el enlace funciona.
- ✓ ☐ Incluí capturas de un error 404 y un error 422 donde se pidió.
- ✓ ☐ Exporté el documento a PDF antes de subirlo.